



Docs

[Distributed tracing](#) >[Errors inbox](#) >[Workloads](#) >[New Relic Lens](#) >

NRQL

[Get started with NRQL](#)[How to query with NRQL](#)[NRQL charts and dashboards](#)[NRQL syntax](#)[NRQL tutorials](#)[Your first queries](#) [Tutorial](#)[Process your data](#) [Tutorial](#)[Advance your dashboarding](#) [Tutorial](#)[Advanced functions](#) [Tutorial](#)[NRQL querying](#)[NRQL examples](#)[Schedule NRQL searches](#)[Service Architecture Intelligence](#) >[Service level management](#) >[Workflow Automation](#) >

SECURITY

[Interactive application security testing \(IAST\)](#) >[Security RX \(Remediation Explorer\)](#) >

PRODUCT UPDATES

[Release notes](#) >[What's new?](#) >[End-of-life announcements](#)

ADMIN AND DATA

[Account & user management](#) >[Data and APIs](#) >[Data dictionary](#)[Security and privacy](#) >[Licenses](#) >

Use advanced NRQL functions

So far, you've learned the basics of using NRQL, how to use NRQL to control your data, and how to use NRQL to advance your dashboarding. In this final tutorial series, you'll learn the most advanced NRQL techniques that enable you to query absolutely anything you need from your data. This tutorial will discuss additional aggregation techniques, higher-level math functions, and advanced features like Regex filtering and nested aggregation. Specifically, you'll learn how to use:

- The `stddev()` aggregation function, and how to group aggregated data using the `buckets()` function.
- The advanced math functions available within NRQL to smooth, clamp and manipulate the data.
- How to discover the event types and attributes available in your data within a specific period.
- How to filter data with Regex using `RLIKE`.
- How to use nested aggregation and subqueries.

We think you'll find these features downright invaluable. Let's get started!

Use bucketing with aggregation

- 1 Standard deviation measures the amount of variation or dispersion within a set of values. It uses a scale from low (values

On this page

[Use bucketing with aggregation](#)[Use advanced math functions](#)[Discover events and attributes](#)[Filter with regex](#)[Use nested aggregation](#)[Use subqueries](#)

Was this doc helpful?

Yes

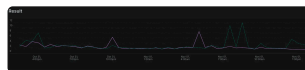
No

Edit this doc

close to the average) to high (values far from the average). The `stddev()` function lets you look between the lines of averages and understand reported values on a deeper level. In this example, you compare the standard deviation of transaction response time ("duration") for the last day to the previous day.

```
SELECT stddev(duration)
FROM Transaction
SINCE 24 hours ago
COMPARE WITH 24 hours ago
TIMESERIES
```

 Copy



The `stdvar()` function works in a similar way to `stddev()` but returns the standard variance for numeric attributes.

2

In a previous lesson, you learned how to group data into a specific configuration of buckets using `FACET cases()`. You can also bucket data by a specific attribute automatically using `FACET buckets()`. This function simplifies grouping data for any aggregation function and takes three arguments:

```
buckets(attribute,
ceiling, number-of-
buckets) .
```

- The attribute you want to bucket by.
- Maximum value of the sample range (any

outliers will appear in the final bucket).

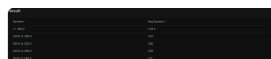
- The total number of buckets you need.

In the example query, you use NRQL to find the average duration of an entire transaction. But, you group that performance into transactions that make specific volumes of database calls. So, you specify the bucket attribute as

`databaseCallCount`, set the ceiling at 400 calls, and group it in 10 buckets. This results in the performance of transactions making 0-40, 40-80, 80-120, 120-160, 160-200, 200-240, 240-280, 280-320, 320-360, and >360 Database Calls. 10 clear buckets divided evenly up to the ceiling.

```
SELECT average(duration)
FROM Transaction
SINCE 12 hours ago
FACET
buckets(databaseCallCount,
400, 10)
```

 Copy



duration	count
0-40	10
40-80	10
80-120	10
120-160	10
160-200	10
200-240	10
240-280	10
280-320	10
320-360	10
>360	10

With this, you've now learned all aggregation functions that currently exist in NRQL! You can slice and dice your data like a pro. If you think an aggregation ability is missing from NRQL, let your account team know: we're always looking for the next killer

functionality that you might need.

Use advanced math functions

NRQL also supports more advanced mathematical functions for those who need to get really deep into their data. These functions can extrapolate values to power of X, find square roots, apply clamping to impose upper and lower bounds, or even just keep values positive on the Y axis.

- 1 NRQL has many math functions that manipulate values in some way. In this example, you'll see a few demonstrated simultaneously.

The `abs(n)` function returns the absolute value for n: for non-negative n values it returns "n", and for negative n values it returns the positive number "n". For example, `abs(2) = 2`, and `abs(-4) = 4`.

You can also round decimal numbers to integers using `floor()`, `ceil()`, and `round()`. `floor()` returns the closest full integer rounding down, and `ceil()` does the opposite by rounding up. `round()` is bi-directional, and rounds up or down to the closest full integer.

```
SELECT abs(duration),  
       round(duration),  
       ceil(duration),  
       floor(duration)  
FROM Transaction  
SINCE 1 day ago
```

 Copy



Transaction Name	Duration	Other Metrics
Transaction 1	10.5	...
Transaction 2	11.2	...
Transaction 3	9.8	...
Transaction 4	12.1	...
Transaction 5	10.0	...

- 2 You can use clamping on an attribute to impose an upper or lower limit on its value. This has use for things like ensuring extreme outliers don't skew the scale of a time series graph.

```
clamp_max(duration, 10)
```

returns the duration, unless it exceeds 10, in which case it returns 10. Quite simply, anything greater than 10 still returns as to equal 10.

```
clamp_min(duration, 1)
```

does the inverse; if any duration is below 1, it reports as equal to 1.

```
SELECT
  clamp_max(average(duration),
    10),
  clamp_min(average(duration),
    1)
FROM Transaction
SINCE 1 day ago
TIMESERIES
```

 Copy



- 3 Now you can move on to the advanced maths that many find challenging to use. As before, we can show you all of these functions in a single example query.

The `pow()` function raises the first argument to the power of the second argument. In this example, you raise duration to the power of 2. If you need the square root of a value, the `sqrt()` function can quickly give this to you. And the

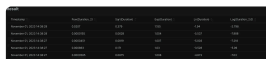
`exp()` function computes the natural exponential function of its argument.

Finally, NRQL also offers logarithms catered to a number of similar functions:

- `ln(n)` computes the natural logarithm: the logarithm base e.
- `log2(n)` computes the logarithm base 2.
- `log10(n)` computes the logarithm base 10.
- `log(n, b)` allows you to compute logarithms with an arbitrary base b.

```
SELECT pow(duration, 2),  
       sqrt(duration),  
       exp(duration),  
       ln(duration),  
       log2(duration)  
FROM Transaction  
SINCE 1 day ago
```

 Copy



duration	pow(duration, 2)	sqrt(duration)	exp(duration)	ln(duration)	log2(duration)
100	10000	10	27071.995	4.60517	6.64386
200	40000	14.1421	134001	5.29832	7.64386
300	90000	17.3205	444729	5.70378	8.20915
400	160000	20	135291	5.99146	8.64386
500	250000	22.3607	371556	6.21461	8.96581

In this lesson, you learned about smoothing out your event data.

`round()` / `ceil()` / `floor()` let you round in whichever manner you like. Clamping lets you put bounds on the data, and the advanced mathematic tools for logarithm, square root, power and exponential all offer further control to manipulate data as you need to. Next, you'll learn about how to discover events and attributes.

Discover events and attributes

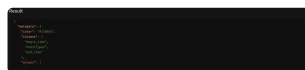
Discovering events and attributes can help answer questions about

your available data before querying it and assist with automation! Let's take a moment to explore this powerful utility.

- 1 Let's say you want a list of all event types currently reporting to your New Relic account. The `SHOW EVENT TYPES` syntax returns a list of all reported event types in a given period. It's one of the rare exceptions where a NRQL query doesn't need `SELECT` and `FROM`. You could use this functionality for things like confirming the existence of custom event data.

```
SHOW EVENT TYPES
SINCE 1 week ago
```

 Copy



-
- 2 You may need to know what attributes you have available for a given event type. The `keyset()` function provides a list of all attributes for an event type, grouped by attribute type. You'll only get attributes that contain values within the provided time window, which you can use to explore your data. You can also use it in automation, for things like ensuring customer data reports correctly.

```
SELECT keyset()
FROM Transaction
SINCE 1 week ago
```

 Copy



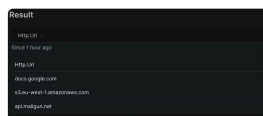
These features help you discover changes in event types and attributes. More DevOps engineers use these functionalities to wrap up jobs, or even quickly automate them. Next, you'll learn how to filter data with Regex.

Filter with regex

- 1 You may occasionally want to filter data with more complex pattern matching than `LIKE` can provide. With the `RLIKE` clause, you can filter with regular expression for more complicated matching.

In any scenario you could use `LIKE`, you have the option to use `RLIKE` instead to provide a corresponding regex in quotations. In this example, there's a list of all host names ending in even numbers or consonants.

```
SELECT uniques(host)
FROM Transaction
WHERE host
RLIKE '^.*[02468bcd fghjklmnpqrstvwxyz]
```



-
- 2 You can also do simple matching. For instance, if you wanted to match a value that starts with a given letter or word.

```
SELECT count(*)  
FROM Transaction  
WHERE name  
RLIKE 'w.*|0.*'  
FACET name
```

 Copy

The regular expression engine uses RE2 syntax. If you need to delete characters, you may need to use the double backslashing escape sequences. (e.g. `\\`).

Remember that `RLIKE` has inherently more complexity and less performance than `LIKE` . Only use it when `LIKE` and other filtering functionality fails to fit your needs.

Regular expression support allows for near-infinite pattern matching possibilities. If you are already a regex guru, you know the power this adds to NRQL. But if you're not, don't worry! Regex has many resources available to reference online. Now that you've learned how to use regex with NRQL, next on the list is using nested aggregation.

Use nested aggregation

You can write nested aggregation queries with NRQL which uses a query as the `FROM` for the parent query. This allows you to answer questions such as:

- How many transactions per minute did my application handle, and what was the maximum rate of requests per minute in the last hour?
- What is the average CPU usage of all my servers, and which

specific servers are over 90%?

- What percentage of all user sessions bounced immediately (i.e. only one PageView in the session)?

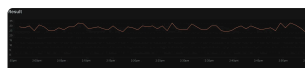
Let's explore each of these use cases in more detail.

1 Example 1: Max API calls for the last hour

First, you can count the number of API calls per minute over the last hour. This returns 60 data points on a graph:

```
SELECT count(*) AS apicalls
FROM Transaction
TIMESERIES 1 MINUTE
```

 Copy



Now, in order to find the maximum value reported across that period, you wrap the query in parentheses, and use `SELECT ... FROM` like this: `SELECT z FROM (SELECT x AS z FROM y)`

```
SELECT max(apicalls)
FROM ( SELECT count(*) AS
apicalls FROM Public_APICall
TIMESERIES 1 minute )
```

 Copy

2 Example 2: servers with high CPU load

This example uses data from New Relic infrastructure. Sometimes, you only want to see hosts whose CPU has, on average, exceeded a certain percentage. If you ask NRQL for the `average(cpuPercent)`, you get a list of hosts with the highest average CPU

percent. But, you can't simply add a `WHERE cpuPercent > 90` to limit this to only hosts running at 90% or above, because this would remove the data before calculating the average.

But you can solve this with nested aggregation! By asking for `average(cpuPercent)` in the sub query, you get the list of hosts and their average CPU. Now, in the outer query, you can filter to only results that were `> x%`! (Tip: You may need to adjust the threshold of this query to work with your hosts' CPU. We've set it to 20% here.)

```
SELECT hostname, cpu
FROM (SELECT
  average(cpuPercent) AS cpu
FROM SystemSample FACET
  hostname)
WHERE cpu > 20
```

 Copy

3

Example 3 - Session Bounces

People often wonder how to calculate the bounce rate on front-end monitoring. This refers to sessions that view a single page, and "bounce" away before visiting more pages. With nested aggregation, you can achieve this easily. Our inner query counts the `PageViews`, grouping them by session. The result set passed to the outer query lists all the sessions and how many pages each viewed. The outer query then calculates the percentage of sessions with a count of 1

(this indicates a "bounced session", because they only viewed a single page).

```
SELECT percentage(count(*),
WHERE sessionLength = 1)
FROM (SELECT count(*) AS
sessionLength FROM PageView
FACET session)
```

 Copy

In this lesson, you've learned how you can use a query as the `FROM` of another query to answer more complicated questions. There's only one final thing to learn: how to use subqueries!

Use subqueries

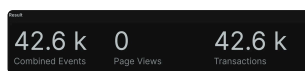
Subqueries in NRQL are similar to nested aggregations, allowing you to use a query nested inside another query. With subqueries, the nested query goes into the `SELECT` statement and the `WHERE` clause, while nested aggregations go into the `FROM` clause.

Let's look at some examples of these different types of subqueries.

1 Numeric Conditions

You can use any query which returns a single numeric value in numeric conditions. This example uses a subquery in the `WHERE` clause that returns the value for the 97th percentile of the duration for API calls, and then returns the name and the duration for those calls greater than that value.

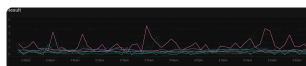
```
SELECT http.url, duration
FROM Public_APICall
WHERE duration > (SELECT
percentile(duration,97) FROM
Public_APICall)
```

 Copy

2 IN Conditions

When you have multiple values returning from a subquery, use an `IN` condition for the parent query to compare against each value. In the example, the `entity.guid` attribute provides data for each unique entity that has a transaction error, and this matches against the `entity.guid` value for the transactions to determine the average duration for the erroring entities.

```
SELECT average(duration)
FROM Transaction
WHERE entity.guid
IN (SELECT
  uniques(entity.guid) FROM
  TransactionError)
FACET appName
TIMESERIES
```

 Copy

3 Subqueries in the SELECT statement

You can use subquery results in calculations with a `SELECT` statement, and may specify a different time range from the outer query. This example calculates the delta between the current average duration and that of the last 7 days.

```
SELECT average(duration) -  
(SELECT average(duration)  
FROM Public_APICall SINCE 7  
days ago)  
FROM Public_APICall  
TIMESERIES
```

 Copy

Subqueries are a powerful tool for data exploration, allowing for more sophisticated queries across different data sources and time ranges.

Congratulations on completing this tutorial. You learned about additional aggregation techniques, higher-level math functions, and advanced features like Regex filtering, nested aggregation and subqueries. If you've completed the previously three tutorials, then you've worked with nearly every aspect of NRQL that you can use!

We hope you'll find NRQL useful for exploring your data, and if you have further questions or encounter problems, feel free to [contact New Relic support](#) . Or, if you'd prefer, you can share your experience with our [online community](#) .

[Careers](#) [Terms of Service](#) [DMCA Policy](#) [Privacy](#) [Website Terms](#)
[Your Privacy Choices](#)  [Patent Notice](#) [UK Slavery Act](#)

COPYRIGHT © 2026 NEW RELIC INC. This site is protected by reCAPTCHA and the Google Privacy Policy and Terms of Service